

Manuel de Réalisation

Détection d'URL Malveillantes par IA

Réalisé par :

MIHI Badis Mohammed Ali
KAID Akram
Sennouci Abdeldjalil Kouider
MAZARI Amine Abdessamed

Année : 1^{er} mai 2026

Table des matières

1	Introduction	2
2	Méthodologie	2
2.1	Prétraitement des Données	2
2.2	Extraction de Caractéristiques	2
2.3	Modèles Implémentés	2
3	Évaluation	2
4	Guide d'Utilisation	3
5	Résultats	3
6	Analyse	3
7	Conclusion	3
8	Démonstration et Test en Temps Réel	3
8.1	Étude de Cas : Test de Phishing	4
8.2	Interprétation	4

1 Introduction

L'objectif de ce projet est de concevoir un système capable de classer les URL en quatre catégories : **benign**, **phishing**, **malware** et **defacement**. Le dataset utilisé est **ISCX-URL2016**.

2 Méthodologie

2.1 Prétraitement des Données

- **Nettoyage** : conversion en minuscules, suppression de `http://`, `https://` et `www.`
- **Encodage** : transformation des labels avec `LabelEncoder`
- **Partitionnement** : split 80% / 20% avec stratification

2.2 Extraction de Caractéristiques

Deux approches combinées :

1. **TF-IDF** : capture des motifs textuels
2. **Features artisanales** :
 - Longueur de l'URL
 - Nombre de caractères spéciaux
 - Présence de @, ?, =
 - HTTPS ou non
 - Ratio chiffres/lettres

2.3 Modèles Implémentés

- Régression Logistique
- Random Forest
- XGBoost
- LSTM

3 Évaluation

- Accuracy
- Precision / Recall / F1-score

- Matrice de confusion
- ROC-AUC

4 Guide d'Utilisation

1. `pip install -r requirements.txt`
2. Placer dataset dans `data/malicious_phish.csv`
3. `python main.py`

5 Résultats

Modèle	Accuracy	Precision	Recall	F1
Random Forest	98.09%	98.08%	98.09%	98.07%
XGBoost	97.40%	97.37%	97.40%	97.36%
LSTM	96.34%	96.30%	96.34%	96.29%
Régression Logistique	91.65%	91.40%	91.65%	91.44%

TABLE 1 – Comparaison des modèles

6 Analyse

Le modèle **Random Forest** est le plus performant, montrant l'efficacité des features artisanales. Le modèle **LSTM** démontre une bonne capacité à apprendre directement à partir des séquences.

7 Conclusion

Ce projet montre que la combinaison de techniques classiques et de deep learning permet d'obtenir d'excellents résultats dans la détection d'URL malveillantes.

8 Démonstration et Test en Temps Réel

Pour valider le fonctionnement du modèle, un script de prédiction en temps réel (`predict.py`) a été développé. Ce script permet de tester n'importe quelle URL via une interface en ligne de commande.

8.1 Étude de Cas : Test de Phishing

Nous avons soumis l'URL suivante au modèle **Random Forest** sauvegardé :

```
http://secure-login-bank.com/auth
```

Résultats obtenus :

- **Classification** : **PHISHING**
- **Confiance** : 71.00%
- **Détails des probabilités** :
 - Benign : 6.00%
 - Malware : 17.00%
 - Defacement : 6.00%
 - **Phishing** : **71.00%**

8.2 Interprétation

Le modèle identifie correctement cette URL comme une tentative de phishing. Cette décision est basée sur plusieurs facteurs clés :

1. **Sémantique** : présence de mots-clés sensibles (*secure, login, bank*)
2. **Protocole** : utilisation de **http** au lieu de **https**
3. **Structure lexicale** : domaine avec plusieurs tirets, typique des attaques